

SISTEMAS OPERATIVOS

08011442

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



Presentación del curso
Sistemas Operativos · 2026-II

Contenido

1. Información del curso
2. Objetivos y resultados de aprendizaje
3. Competencias por desarrollar
4. Justificación y ubicación en la malla curricular
5. Contenido por unidades
6. Planificación por semanas
7. Metodología, prácticas y lecturas
8. Esquema de evaluación
9. Bibliografía
10. Repositorio del curso

Información del curso

Asignatura	Sistemas Operativos (código 08011442) · Área de Ingeniería Aplicada
Programa	Ingeniería de Sistemas · Facultad de Ingeniería · Universidad del Magdalena
Créditos	4 créditos · 64 h de acompañamiento directo (HADD) y 128 h de trabajo independiente (HTI), proporción 1:2
Prerrequisito	Arquitectura de Computadores
Correquisito	Estructuras de Datos
Grupos	Grupos 1, 2 y 3 (periodo 2026-II) · cada estudiante asiste en el horario del grupo matriculado
Docente	Germán Sánchez Torres, I.S., M.Sc., Ph.D. · gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com · Edificio Docente, cubículo 3D401

Reglas de juego: no se asiste en un horario distinto al matriculado; cada estudiante responde por los exámenes y talleres programados para su grupo; el planificador de actividades y el microdiseño son de obligatorio cumplimiento.

Objetivos del curso

Objetivo general

Ofrecer una descripción de los fundamentos de diseño, los conceptos algorítmicos y las estructuras que sustentan el funcionamiento de los sistemas operativos.

Objetivos específicos

- Definir qué es un sistema operativo y los servicios que debe proporcionar.
- Diferenciar los elementos de un sistema operativo de propósito general construido sobre el modelo de von Neumann.
- Capacitar al estudiante como usuario y como programador del sistema.
- Manejar las facilidades POSIX para crear, comunicar y sincronizar procesos e hilos.
- Apropiar los conceptos de administración de memoria principal y del modelo de memoria virtual.

Resultados de aprendizaje del curso

CLO	Al finalizar el curso, el estudiante estará en capacidad de…	SO
CLO1	Explicar y justificar los servicios de un sistema operativo y su organización estructural (monolítica, por capas o de micronúcleo) dentro del modelo de von Neumann.	SO1
CLO2	Modelar e implementar procesos e hilos; aplicar la comunicación entre procesos (tuberías, memoria compartida, colas de mensajes) y analizar la sincronización y la planificación de la CPU con evidencia medida.	SO1, SO2, SO6
CLO3	Aplicar y evaluar esquemas de gestión de memoria y de sistemas de archivos mediante experimentos controlados.	SO2, SO6
CLO4	Diseñar, implementar y evaluar programas en C/POSIX que integren procesos, comunicación entre procesos, sincronización y memoria, y reportar los métodos y los resultados experimentales.	SO1, SO2, SO6

CLO: resultados de aprendizaje del curso (syllabus ABET). SO: resultados del estudiante del programa a los que aporta cada uno; se describen en la diapositiva siguiente.

Competencias por desarrollar

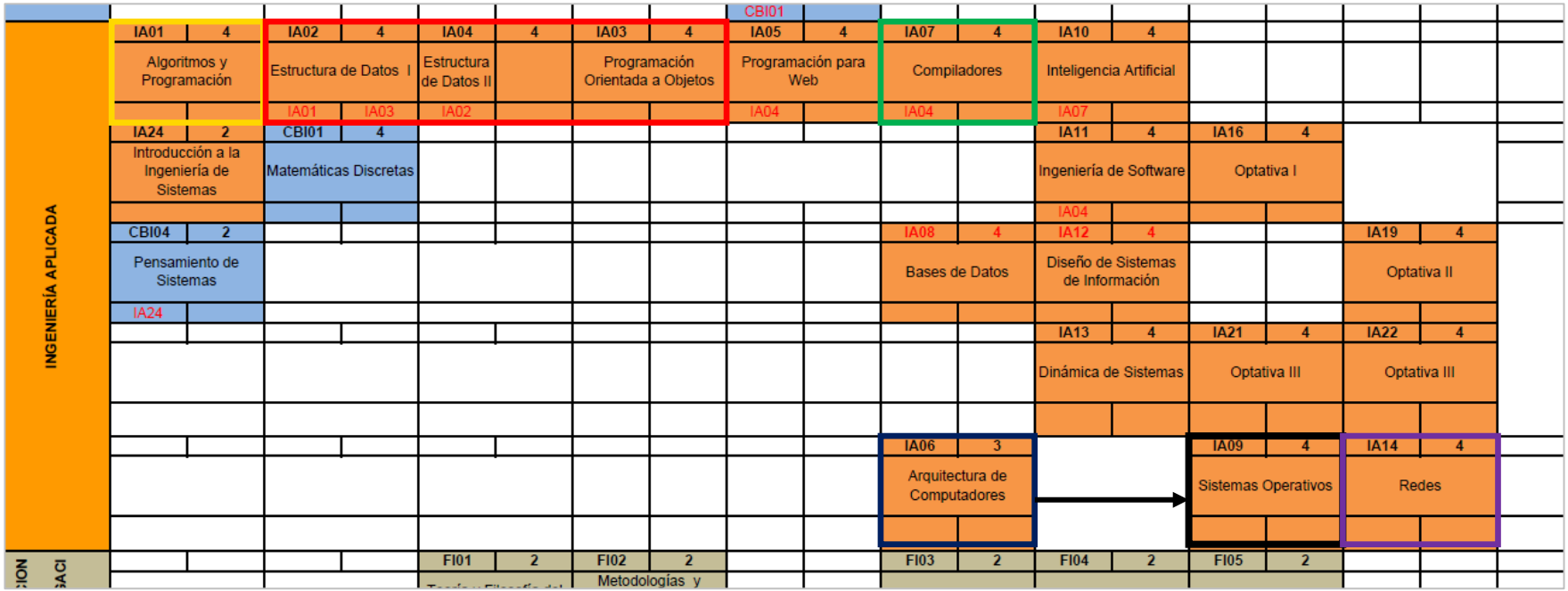
Tipo	Competencias del microdiseño
Genéricas	Conocimiento general básico · capacidad de análisis y síntesis · resolución de problemas · habilidades de investigación · compromiso ético.
Específica	Identificar y apropiar los principios básicos del funcionamiento de los sistemas operativos para desarrollar programas de computador más estables y eficientes.

SO	Resultados del estudiante (ABET) a los que aporta el curso
SO1	Identificar, formular y resolver problemas complejos de ingeniería aplicando principios de ingeniería, ciencias y matemáticas.
SO2	Aplicar el diseño de ingeniería para producir soluciones que satisfagan necesidades específicas, considerando la salud pública, la seguridad y el bienestar, y factores globales, culturales, sociales, ambientales y económicos.
SO6	Desarrollar y conducir experimentación apropiada, analizar e interpretar datos y usar el juicio de ingeniería para extraer conclusiones.

¿Por qué un curso de sistemas operativos?

- «Los sistemas operativos son una parte esencial de cualquier sistema de computador; por lo tanto, un curso sobre sistemas operativos es parte fundamental de cualquier programa en ciencias de la computación» (Silberschatz).
- El curso trata los temas generales, básicos y transversales a todas las arquitecturas: profundizar en cada uno desbordaría un solo semestre.
- Cada tópico se enmarca en el estándar **POSIX** (IEEE 1003.1 e ISO 9945-1), que unifica las interfaces de los distintos sistemas operativos.
- Naturaleza práctica: se busca el balance entre teoría y práctica. Al terminar, el estudiante debe dominar los conceptos y saber usarlos e implementarlos en programas reales.
- Resultado esperado: programas más estables y eficientes gracias a entender cómo funciona el sistema por debajo.

Ubicación en la malla curricular



Algoritmos y programación
Estructuras de datos y POO
Compiladores
Arquitectura de Computadores
Sistemas Operativos (IA09)
Redes

La flecha indica el prerrequisito (Arquitectura de Computadores → Sistemas Operativos). Fuente: malla curricular del programa de Ingeniería de Sistemas.

Contenido por unidades

Unidad	Temas	HADD	HTI	Total h
1. Introducción a los sistemas operativos	• 1.1 Conceptos arquitectónicos • 1.2 Jerarquía de memoria • 1.3 Evolución de los sistemas operativos	12	28	40
2. Procesos y comunicación entre procesos	• 2.1 Multitarea y multiprogramación • 2.2 Creación, estados y planificación de procesos • 2.3 Señales • 2.4 Esquemas y persistencia de las herramientas de comunicación • 2.5 Tuberías, memoria compartida y colas de mensajes	24	46	70
3. Sincronización de la ejecución	• 3.1 Hilos • 3.2 Sincronización entre hilos: semáforos • 3.3 Mutex y variables de condición	16	28	44
4. Gestión de memoria	• 4.1 Modelo de memoria de un proceso • 4.2 Esquemas de memoria • 4.3 Memoria virtual	12	26	38

Horas según el microdiseño: 64 h de acompañamiento directo (HADD) y 128 h de trabajo independiente (HTI); total 192 h, 4 créditos.

Planificación por semanas (1 de 2)

Semana	Temas y subtemas	Metodología	Trabajo autónomo del estudiante
1	Presentación e introducción a la cátedra. Conceptos arquitectónicos del computador	Socialización de la metodología de evaluación; evaluación inicial y entrega de material	Lecturas: Lecture 01 (Unix time-sharing system) y Lecture 02 (eficiencia energética de los lenguajes)
2	Introducción a los sistemas operativos	Presentación de la temática	Unidad 1 del libro guía (Silberschatz); Lecture 03 (working set)
3	Procesos y planificación de procesos	Presentación de la temática; taller práctico	Unidad en el texto guía
4	API de procesos (POSIX) y planificación	Presentación de la temática; taller práctico	Unidad en el texto guía
5	Talleres	Talleres individuales y grupales; sustentación	
6	Comunicación entre procesos: señales	Presentación de la temática	Unix Network Programming: unidad I y capítulo II, parte III
7	Talleres	Taller práctico	
8	Comunicación mediante tuberías	Presentación de la temática	Ejemplos prácticos

Fuente: formato de planeación de actividades DO-F49. Las fechas exactas se publican en la plataforma del curso.

Planificación por semanas (2 de 2)

Semana	Temas y subtemas	Metodología	Trabajo autónomo del estudiante
9	Talleres	Talleres individuales y grupales	
10	Memoria compartida y colas de mensajes (System V IPC)	Presentación de la temática; taller práctico	Unix Network Programming, vol. 2: capítulos 6 y 14
11	Talleres	Talleres individuales y grupales	
12	Hilos de ejecución	Presentación de la temática	Trabajo práctico de laboratorio: desarrollo de la guía
13	Hilos de ejecución	Taller práctico	Desarrollo de la guía de laboratorio; Lecture 04 (memoria virtual)
14	Talleres y evaluación parcial II	Talleres individuales y grupales; sustentación	
15	Talleres prácticos	Talleres individuales y grupales; sustentación	
16	Semana de recuperación		

Fuente: formato de planeación de actividades DO-F49. Las fechas exactas se publican en la plataforma del curso.

Metodología y prácticas de laboratorio

- **Clases magistrales:** exposición de los principios y contenidos de cada tema según la programación.
- **Laboratorios:** desarrollo de programas que aplican las técnicas vistas, guiados por una guía metodológica con el problema, el esquema de solución y las técnicas a usar.
- **Lecturas de artículos científicos:** lecturas obligatorias con evaluación (ver esquema de evaluación).
- **Talleres individuales y grupales** con sustentación, y coevaluación en las sesiones de laboratorio.

Práctica	Tema	Semana
Introducción al lenguaje C	Conceptos arquitectónicos	1
fork y exec	Procesos	3
Pthreads	Hilos de ejecución	5
pipe y memoria compartida	Tuberías y memoria compartida	6

Prácticas académicas del microdiseño (2 h en laboratorio cada una). Las semanas indicadas son las del microdiseño; la planificación semanal vigente puede reubicarlas.

Lecturas del curso

Seguimiento	Artículo	Semana
1	**Lecture 01:** The Unix Time-Sharing System (Ritchie y Thompson)	2
1	**Lecture 02:** Energy Efficiency across Programming Languages (Pereira et al.)	3
2	**Lecture 03:** The Working Set Model for Program Behavior (Denning)	7
3	**Lecture 04:** Virtual Memory Benefits and Uses (Chaplot)	14
Complementarias	Operating System and Artificial Intelligence: A Systematic Review (Zhang et al.) · The End of Moore's Law (Theis y Wong) · The Gap between Processor and Memory Speeds (Carvalho)	

Los artículos están en la carpeta `lectures/` del repositorio del curso, con el mismo número (Lecture 01 a 04). Cada lectura obligatoria se evalúa en el seguimiento indicado.

Esquema de evaluación

Seg.	Actividad	Temas a evaluar	Semana	Puntaje
1	Evaluación de presaberes	Diagnóstico inicial	1	NA
1	Lecturas de artículos y taller práctico	Lecture 01 y 02; creación de procesos y API POSIX de procesos	2, 3 y 5	10 %
1	Examen parcial teórico	Conceptos arquitectónicos, introducción a los SO, procesos y planificación	6	20 %
2	Lectura y talleres prácticos en C sobre Linux	Lecture 03; señales; comunicación mediante tuberías	7, 9 y 10	15 %
2	Examen parcial práctico	Comunicación entre procesos: señales, tuberías, memoria compartida y colas de mensajes	12	20 %
3	Lectura y taller práctico	Lecture 04 (memoria virtual); programas multihilo	14 y 15	15 %
3	Examen parcial práctico	Hilos de ejecución y sincronización	15	20 %

Seguimiento 1: 30 % · Seguimiento 2: 35 % · Seguimiento 3: 35 %. Fuente: formato de parámetros de evaluación DO-F50. Las pruebas escritas combinan selección múltiple, interpretación de código y desarrollo de soluciones algorítmicas.

Bibliografía

- **Libro guía:** Silberschatz, Galvin y Gagne. *Operating System Concepts*, 10.ª ed., Wiley, 2018.
- Carretero, García, de Miguel y Pérez. *Sistemas operativos: una visión aplicada*, 2.ª ed., McGraw-Hill, 2007.
- Tanenbaum y Bos. *Sistemas operativos modernos*, 4.ª ed., Pearson, 2015.
- Stallings. *Operating Systems: Internals and Design Principles*, 9.ª ed., Pearson, 2018.
- Arpaci-Dusseau. *Operating Systems: Three Easy Pieces* (OSTEP), 2018. Gratuito en ostep.org.
- Kerrisk. *The Linux Programming Interface*, No Starch Press, 2010.
- Stevens y Rago. *Advanced Programming in the UNIX Environment*, 3.ª ed., Addison-Wesley, 2013.
- Stevens. *UNIX Network Programming*, *vol. 2: Interprocess Communications*, 2.ª ed., Prentice Hall, 1999.
- Kernighan y Ritchie. *The C Programming Language*, 2.ª ed., Prentice Hall, 1988.

Repositorio del curso

https://github.com/sanchezgt/ssoo

```
$ git clone https://github.com/sanchezgt/ssoo.git
$ cd ssoo/example_codes/03-pipes
$ gcc -Wall -Wextra -o pipe1 pipe1.c && ./pipe1
```

`courseinfo/`	microdiseño, planeación y parámetros de evaluación
`presentations/`	las presentaciones del curso en PDF
`example_codes/`	ejemplos de laboratorio por tema, en C
`guides/`	guías de laboratorio
`lectures/`	artículos de lectura obligatoria y complementaria

Escanea para abrir el repositorio

`example_codes/` contiene los ejemplos de laboratorio por tema: `01-c`, `02-fork`, `03-pipes`, `04-signal`, `05-shm`, `06-pthreads` y `07-syncronization`. Se compilan con `gcc -Wall -Wextra` (los de hilos, además, con `-pthread`).